

SLIDE 1 — What is the Address Operator?

Definition:

The **address operator** `&` is used to find the **memory address** of a variable.

Simple idea

Variable → `&` → Memory Address

Example

```
int x = 10;
```

```
printf("%p", (void*)&x);
```

Explanation

- `x` stores the value `10`.
- `&x` gives the **address where `x` is stored**.
- `%p` is used to display a pointer/address.

Example Output

0x7ffd23a4



The actual address will be different on each run/system.



Remember:

`&` means “address of”.

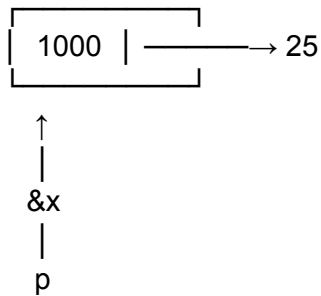
SLIDE 2 — How `&` Works

Suppose:

```
int x = 25;  
int *p = &x;
```

Think of memory like this:

Address	Value
---------	-------



So:

&x → address of x
p → same address
*p → value at that address

Example

```
printf("Address = %p\n", (void*)&x);  
printf("Value = %d\n", x);
```

 **Remember:**

&x does **not** give 25; it gives the **location of x**.

SLIDE 3 — & with Pointers

The address operator is commonly used to **initialize a pointer**.

Example

```
int num = 50;  
int *ptr;  
  
ptr = &num;  
  
printf("Value = %d", *ptr);
```

Output

Value = 50

Step-by-step

num = 50
↓
&num
↓
address of num
↓
ptr stores address
↓
*ptr
↓
50

Key Relationship

ptr == &num

*ptr == num

when **ptr** points to **num**.



Remember:

& connects a variable to its pointer.

SLIDE 4 — **&** in **scanf()**

One of the most important uses of **&** is with **scanf()**.

Example

```
int age;
```

```
printf("Enter age: ");
```

```
scanf("%d", &age);
```

```
printf("Age = %d", age);
```

Why **&age**?

scanf() needs the **address of age** so it knows where to store the input.

User enters 20



scanf()
↓
&age
↓
Stores 20 in age

Output

Enter age: 20
Age = 20

⚠ For a normal `int` variable:

`scanf("%d", &age);` // ✅

not:

`scanf("%d", age);` // ❌

🧠 **Remember:**

`scanf()` usually needs the address of the variable.

SLIDE 5 — Address Operator Recap

Flow

Variable
↓
&
Address
↓
Pointer
↓
*
Value

5 Things to Remember

1. `&` is the **address-of operator**.
2. `&x` gives the address of `x`.
3. It is commonly used to initialize pointers.
4. `scanf()` uses `&` to receive a variable's address.

5. `&` and `*` have opposite roles in basic pointer operations.

Final Example

```
int x = 100;
int *p = &x;

printf("Address: %p\n", (void*)&x);
printf("Value: %d", *p);
Address: [memory address]
Value: 100
```

Memory Trick:

`&` → “Where is it?”

`*` → “What is there?”



QUESTIONS

Q1. MCQ

What does `&x` represent?

- A) Value of `x`
- B) Address of `x`
- C) Pointer value only
- D) Size of `x`

 **Hint:** `&` is the address-of operator.

 **Glossary:** Address → memory location of a variable.

 **Answer: B**

Q2. Fill in the blank

Complete the pointer initialization:

```
int n = 25;
```

```
int *p = _____;
```

 **Hint:** The pointer needs the address of `n`.

 **Glossary:** Initialization → assigning an initial value.

 **Answer:** `&n`

Q3. What is the output?

```
int x = 40;  
int *p = &x;
```

```
printf("%d", *p);
```

- A) Address of `x`
- B) 40
- C) 0
- D) Error

 **Hint:** `*p` accesses the value.

 **Glossary:** Dereference → accessing the value through a pointer.

 **Answer:** B) 40

Q4. Why is `&age` used here?

```
scanf("%d", &age);
```

- A) To get the value of age
- B) To get the address of age
- C) To delete age
- D) To declare age

 **Hint:** `scanf()` needs to know where to store the input.

 **Glossary:** `scanf()` → reads input from the user.

 **Answer:** B

Q5. True or False

`&` gives the value stored inside a variable.

 **Hint:** Compare `&` with `*`.

 **Glossary:** Address-of → obtains a variable's memory address.

 **Answer: False**

Q6. What does this mean?

```
int *p = &x;
```

- A) `p` stores the value of `x`
- B) `p` stores the address of `x`
- C) `x` stores the address of `p`
- D) `p` becomes `x`

 **Hint:** `&x` produces an address.

 **Glossary:** Pointer → variable that stores an address.

 **Answer: B**

Q7. Arrange the steps

For:

```
int x = 10;  
int *p = &x;  
printf("%d", *p);
```

Arrange:

- A. Dereference `p`
- B. Create variable `x`
- C. Get address using `&`
- D. Store address in `p`

 **Hint:** Start with creating the variable.

 **Glossary:** Dereference → access the value at the stored address.

 **Answer:** $B \rightarrow C \rightarrow D \rightarrow A$